

Name: - Sk. Fawaz Ahamad

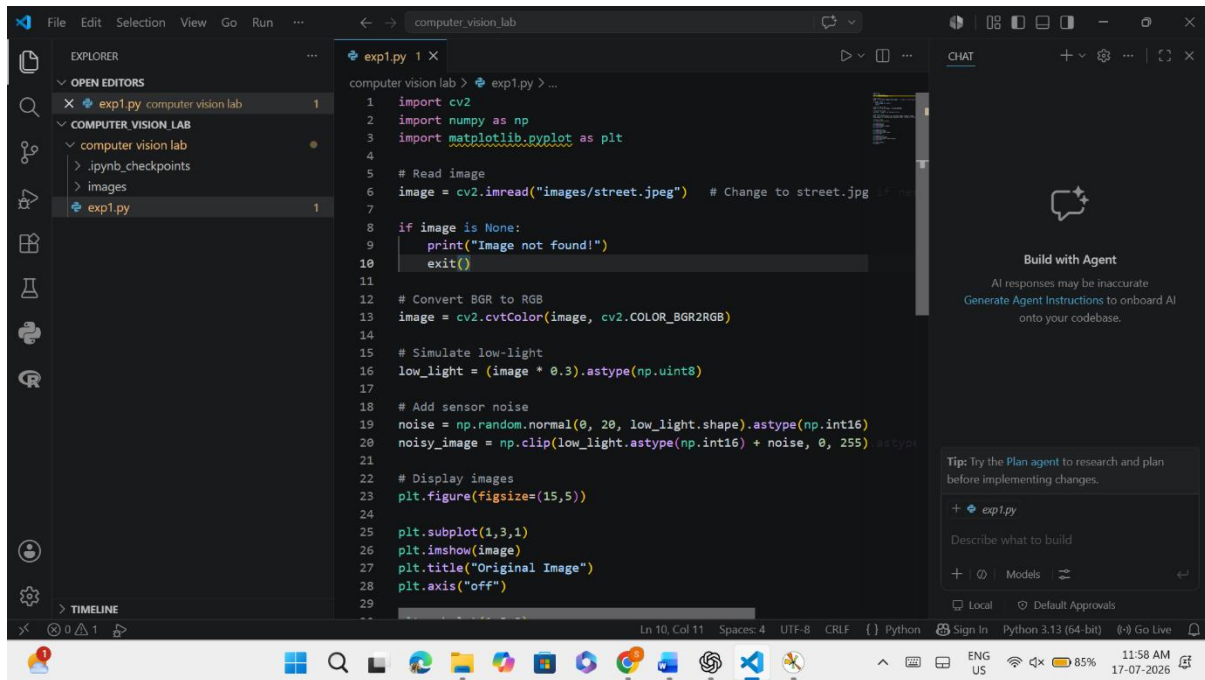
Crouse code: - ITA0517

Reg no: - 192425390

course name: - Computer Vision

2. Given a real-world scene captured under low light, describe how the image sensing and acquisition process influences the final digital image.

Program: -



The screenshot shows a code editor with a file explorer on the left and a chat panel on the right. The main editor displays a Python script named `exp1.py` with the following code:

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Read image
6 image = cv2.imread("images/street.jpeg") # Change to street.jpg if not
7
8 if image is None:
9     print("Image not found!")
10    exit()
11
12 # Convert BGR to RGB
13 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
14
15 # Simulate low-light
16 low_light = (image * 0.3).astype(np.uint8)
17
18 # Add sensor noise
19 noise = np.random.normal(0, 20, low_light.shape).astype(np.int16)
20 noisy_image = np.clip(low_light.astype(np.int16) + noise, 0, 255).astype(
21
22 # Display images
23 plt.figure(figsize=(15,5))
24
25 plt.subplot(1,3,1)
26 plt.imshow(image)
27 plt.title("Original Image")
28 plt.axis("off")
29
```

The chat panel on the right contains the following text:

Build with Agent
AI responses may be inaccurate.
Generate Agent Instructions to onboard AI onto your codebase.

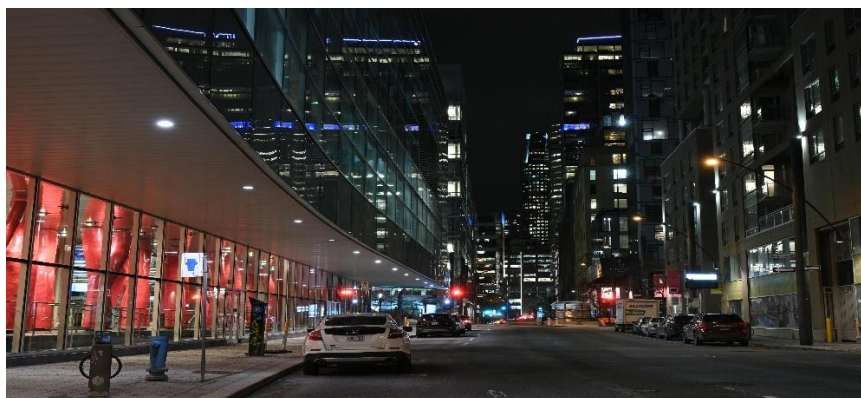
Tip: Try the Plan agent to research and plan before implementing changes.

+ `exp1.py`
Describe what to build

+ | | Models | |

Local | Default Approvals

Input:-



- A street at night
- A road with vehicles
- Buildings with streetlights
- A city scene

Output: -



Result:

The low-light image was successfully generated, demonstrating the effects of image sensing and acquisition on the final digital image.

4. Differentiate how pixel resolution and intensity resolution affect image representation in a practical application like medical imaging.

Program: -

```
File Edit Selection View Go Run ... computer_vision_lab
EXPLORER
  OPEN EDITORS
    exp1.py computer vision lab 1
    exp2.py computer vision lab 1
  COMPUTER VISION LAB
    computer vision lab
      .ipynb_checkpoints
      images
      exp1.py 1
      exp2.py 1
  TIMELINE

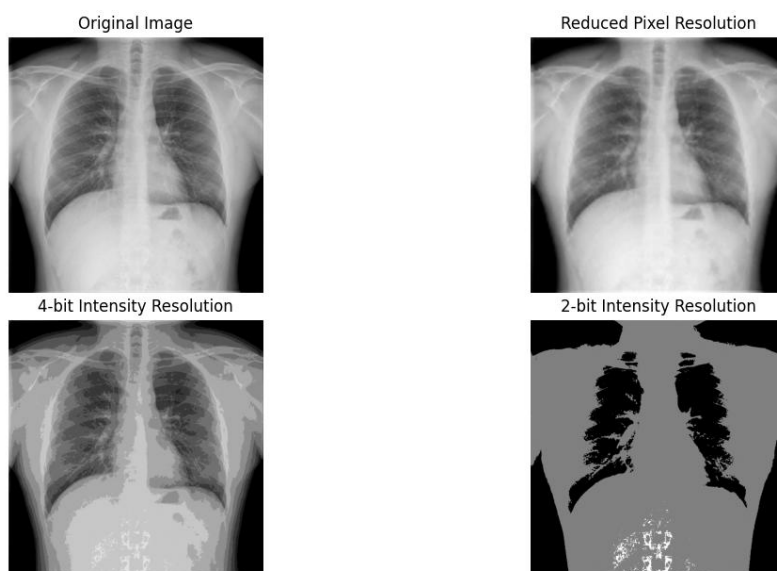
computer vision lab > exp2.py > ...
1 import cv2
2 import matplotlib.pyplot as plt
3
4 # Read the image in grayscale
5 image = cv2.imread("images/medical.jpeg", cv2.IMREAD_GRAYSCALE)
6
7 if image is None:
8     print("Image not found!")
9     exit()
10
11 # -----
12 # Pixel Resolution
13 # -----
14 low_resolution = cv2.resize(image, (128, 128))
15 high_resolution = cv2.resize(low_resolution, (512, 512))
16
17 # -----
18 # Intensity Resolution
19 # -----
20 intensity_4bit = (image // 16) * 16 # 16 Gray Levels (4-bit)
21 intensity_2bit = (image // 64) * 64 # 4 Gray Levels (2-bit)
22
23 # -----
24 # Display Results
25 # -----
26 plt.figure(figsize=(12,8))
27
28 plt.subplot(2,2,1)
29 plt.imshow(image, cmap='gray')
```

Input: -



A grayscale medical image (e.g., Chest X-ray/MRI/CT scan) stored as medical.jpeg.

Output: -

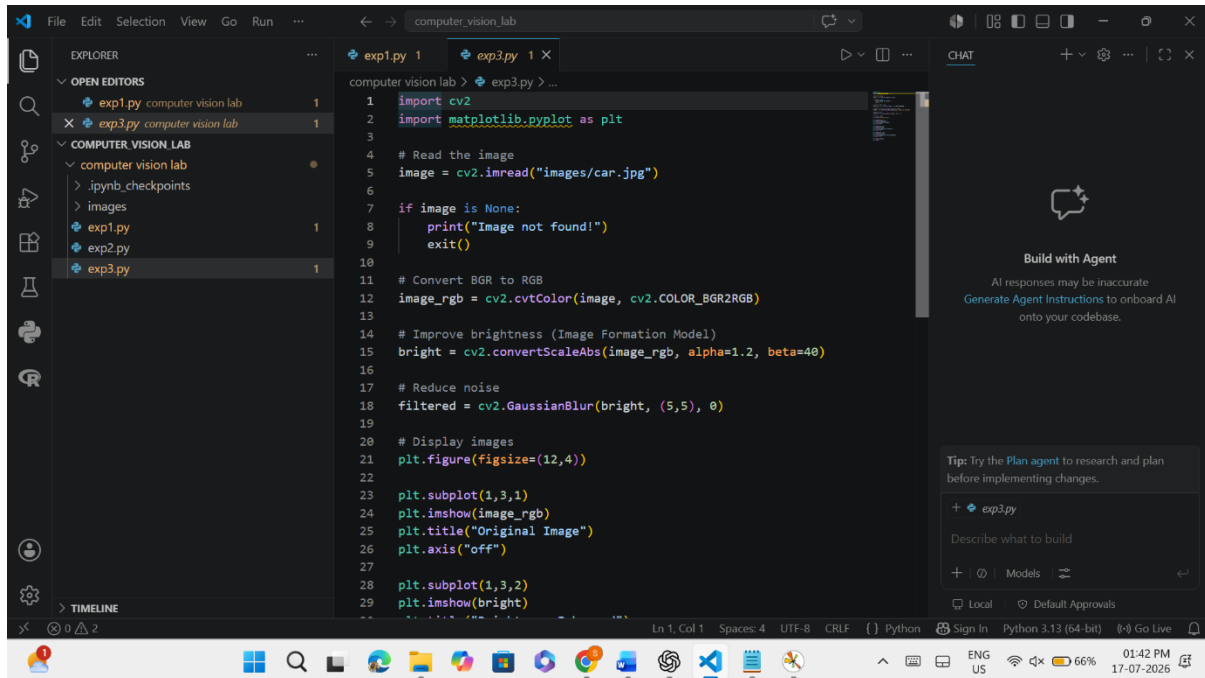


Result:

The program successfully demonstrates the effects of pixel resolution and intensity resolution on image quality.

6. Explain how image formation models help in improving image quality in applications such as autonomous driving.

Program: -



```
1 import cv2
2 import matplotlib.pyplot as plt
3
4 # Read the image
5 image = cv2.imread("images/car.jpg")
6
7 if image is None:
8     print("Image not found!")
9     exit()
10
11 # Convert BGR to RGB
12 image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
13
14 # Improve brightness (Image Formation Model)
15 bright = cv2.convertScaleAbs(image_rgb, alpha=1.2, beta=40)
16
17 # Reduce noise
18 filtered = cv2.GaussianBlur(bright, (5,5), 0)
19
20 # Display images
21 plt.figure(figsize=(12,4))
22
23 plt.subplot(1,3,1)
24 plt.imshow(image_rgb)
25 plt.title("Original Image")
26 plt.axis("off")
27
28 plt.subplot(1,3,2)
29 plt.imshow(bright)
```

Input: -



Road scene image (car.jpg)

Output: -



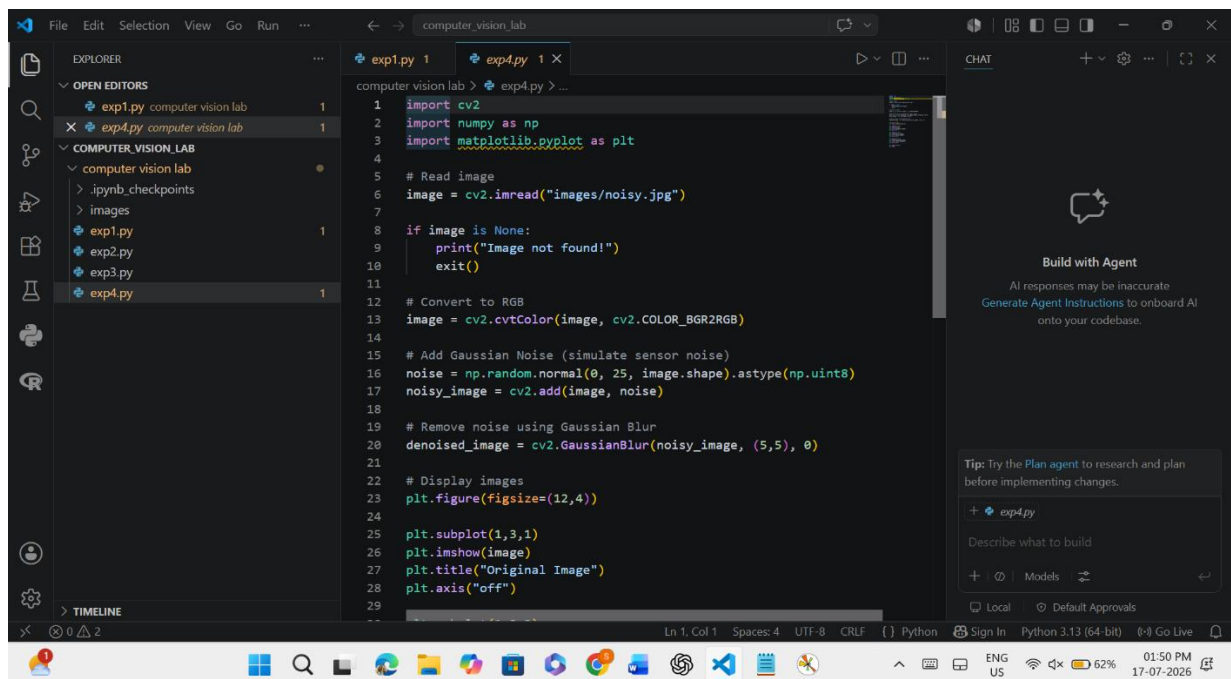
- Original Image
- Brightness Enhanced Image
- Enhanced Image with Noise Reduction

Result:

The program successfully improves the image quality by enhancing brightness and reducing noise for autonomous driving applications.

8. Given a noisy image acquired from a sensor, explain how the acquisition process contributes to noise and how it can be minimized.

Program: -



```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Read image
6 image = cv2.imread("images/noisy.jpg")
7
8 if image is None:
9     print("Image not found!")
10    exit()
11
12 # Convert to RGB
13 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
14
15 # Add Gaussian Noise (simulate sensor noise)
16 noise = np.random.normal(0, 25, image.shape).astype(np.uint8)
17 noisy_image = cv2.add(image, noise)
18
19 # Remove noise using Gaussian Blur
20 denoised_image = cv2.GaussianBlur(noisy_image, (5,5), 0)
21
22 # Display images
23 plt.figure(figsize=(12,4))
24
25 plt.subplot(1,3,1)
26 plt.imshow(image)
27 plt.title("Original Image")
28 plt.axis("off")
29
```

Input: -



Natural scene image (noisy.jpg)

Output: -



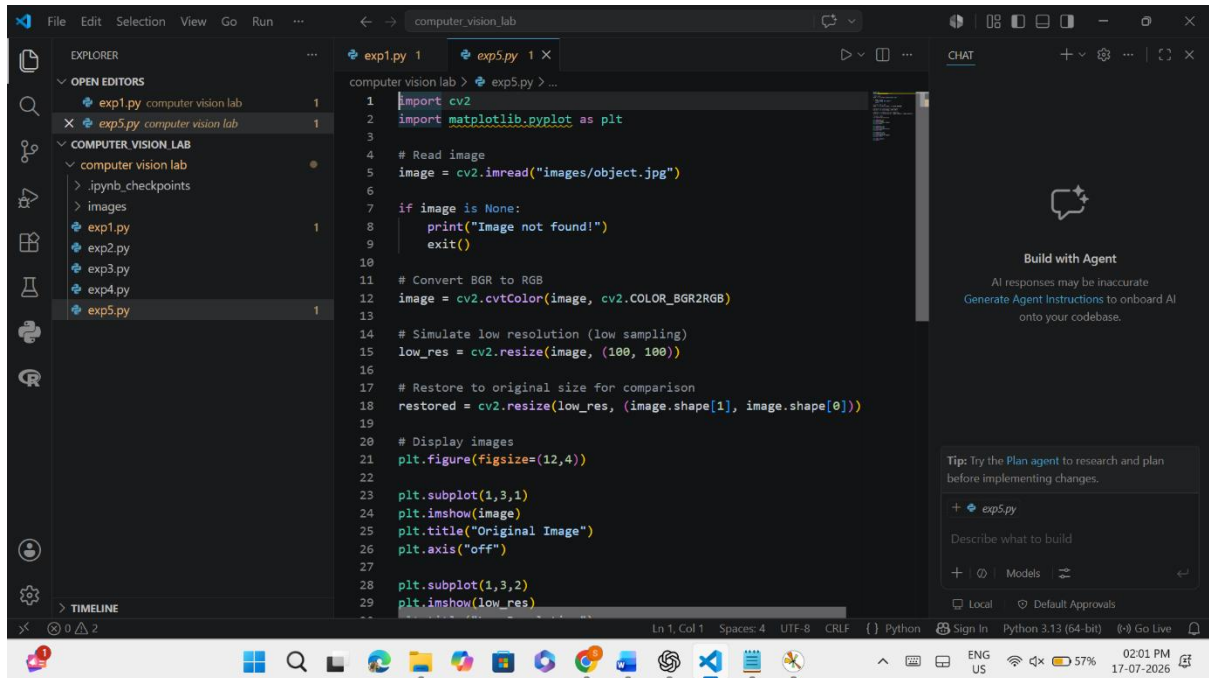
- Original Image
- Noisy Image
- Noise Reduced Image

Result:

The program successfully demonstrates sensor noise and reduces it using Gaussian filtering to improve image quality.

10. An image captured with low resolution fails to detect objects clearly. Explain how sampling and image formation affect object visibility.

Program: -



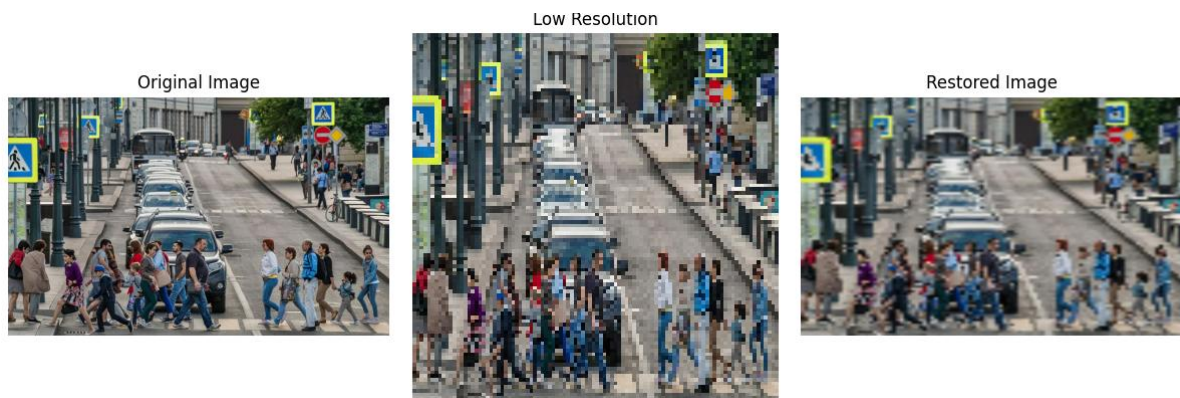
```
1 import cv2
2 import matplotlib.pyplot as plt
3
4 # Read image
5 image = cv2.imread("images/object.jpg")
6
7 if image is None:
8     print("Image not found!")
9     exit()
10
11 # Convert BGR to RGB
12 image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
13
14 # Simulate low resolution (low sampling)
15 low_res = cv2.resize(image, (100, 100))
16
17 # Restore to original size for comparison
18 restored = cv2.resize(low_res, (image.shape[1], image.shape[0]))
19
20 # Display images
21 plt.figure(figsize=(12,4))
22
23 plt.subplot(1,3,1)
24 plt.imshow(image)
25 plt.title("Original Image")
26 plt.axis("off")
27
28 plt.subplot(1,3,2)
29 plt.imshow(low_res)
```

Input: -



Road scene image (object.jpg)

Output: -



- Original Image
- Low Resolution Image
- Restored Image (after resizing)

Result: -

The program successfully demonstrates that low sampling reduces image resolution, making objects less visible and harder to detect.